

PROGRAMAÇÃO ORIENTADA A OBJETOS

2º Semestre

Atividade de Revisão

Sistema de Pedidos

Tema: Encapsulamento · Herança · Polimorfismo · Interfaces · Composição · Arquivos

Objetivo: Consolidar os pilares de POO por meio de um sistema prático de pedidos

Instruções: Implemente e teste cada questão de forma incremental antes de avançar

Contexto

Você irá implementar um sistema simples de pedidos para uma loja. O sistema envolve pessoas, usuários, produtos e pedidos, com aplicação progressiva dos pilares de POO estudados em aula. Leia todos os itens antes de começar e observe como as classes se relacionam.



Questão 1 — Classe Pessoa

Encapsulamento

Crie uma classe chamada Pessoa com os atributos protegidos: cpf, nome, email e senha, todos do tipo String.

- a) **a)** Implemente um construtor que receba e inicialize todos os atributos.
- b) **b)** Implemente os métodos getters e setters para cada atributo.
- c) **c)** Implemente o método mostrar() que exiba o nome da pessoa no console.



Dica: O modificador protected garante que subclasses possam acessar os atributos diretamente — pense por que isso é útil para a classe Usuario.



Questão 2 — Classe Produto

Atributos e Construtor Padrão

Crie uma classe chamada Produto com os atributos: id (int), desc (String) e preco (Double).

- d) **a)** Implemente um construtor padrão (sem parâmetros) que inicialize id com 0, desc com "" e preco com 0.0.
- e) **b)** Implemente o método mostrar() que exiba a descrição do produto no console.

⚠ **Atenção:** Na Questão 6, Produto será transformada em classe abstrata. Já pense em como isso afetará seu código.

Questão 3 — Classe Usuario

Herança

Crie uma classe Usuario que herde de Pessoa e adicione o atributo anoNascimento do tipo int.

- f) **a)** Implemente o construtor de Usuario recebendo todos os dados necessários e utilize `super()` para reaproveitar o construtor da classe pai.
- g) **b)** Explique com suas palavras: por que Usuario não precisa redeclarar os atributos cpf, nome, email e senha?

💡 **Dica:** Lembre-se da sintaxe: `public class Usuario extends Pessoa { ... }`

Questão 4 — Interface Registravel

Interfaces — NOVO ★

Antes de implementar a classe Pedido, crie uma interface chamada Registravel que represente o contrato de qualquer entidade que possa ser registrada em arquivo.

- h) **a)** Declare na interface o método `registrar()` do tipo void, sem implementação.
- i) **b)** Declare também o método `String gerarResumo()`, que retorna um texto representando o objeto.
- j) **c)** Explique com suas palavras: qual é a diferença entre uma interface e uma classe abstrata? Em que situação você escolheria uma em vez da outra?

🎯 **Conceito-chave:** Uma interface define um contrato — ela diz O QUE uma classe deve fazer, mas não COMO. Qualquer classe que implemente Registravel é obrigada a fornecer os dois métodos declarados.

Questão 5 — Classe Pedido

Composição, ArrayList e Interface

Crie uma classe Pedido que implemente a interface Registravel, com os atributos: id (int), usuario (Usuario), produtos (ArrayList<Produto>) e valorPedido (Double).

- k) **a)** Implemente o construtor recebendo um Usuario, um id e um valor inicial, inicializando o ArrayList vazio.
- l) **b)** Implemente `adicionarProduto(Produto p)`: adiciona o produto à lista e soma seu preço ao valor total.

- m) **c)** Implemente `removerProduto(Produto p)`: remove o produto da lista e subtrai seu preço do valor total.
- n) **d)** Implemente `listarProdutos()`: percorre a lista e chama `mostrar()` em cada produto.
- o) **e)** Implemente `consultarProduto(int id)`: busca e retorna um produto pelo seu id, retornando null caso não encontrado.
- f) (Sobrecarga)** Implemente uma segunda versão de `removerProduto` que receba um `int id` e utilize `consultarProduto()` internamente. Qual conceito de POO é aplicado aqui?
- p) **g)** Implemente `registrar()` (da interface): grave os dados em `pedido.txt` usando `FileWriter` e `BufferedWriter` em modo `append`. Formato: `id_pedido;cpf_usuario;id_produto;preco_produto`. Trate exceções com `try/catch`.
- q) **h)** Implemente `gerarResumo()` (da interface): retorne uma `String` com id do pedido, nome do usuário e valor total. Exemplo: "Pedido #1 | Cliente: toim | Total: R\$ 5.0"

 **Dica:** A assinatura da classe ficará: `public class Pedido implements Registravel { ... }`
Se você não implementar todos os métodos da interface, o compilador Java acusará erro.

Questão 6 — Polimorfismo

Herança + Sobrescrita — NOVO 

Agora você vai estender o sistema com dois novos tipos de produto, aplicando polimorfismo.

- r) **a)** Transforme `Produto` em uma classe abstrata e torne o método `mostrar()` abstrato.
- b)** Crie a classe **ProdutoFisico** que herde de `Produto` e adicione o atributo `pesoKg` (`Double`). Sobrescreva `mostrar()` exibindo: `"[FÍSICO] desc - R$ preco (peso: pesoKg kg)"`
- c)** Crie a classe **ProdutoDigital** que herde de `Produto` e adicione o atributo `linkDownload` (`String`). Sobrescreva `mostrar()` exibindo: `"[DIGITAL] desc - R$ preco (download: linkDownload)"`
- s) **d)** No método `main`, crie um `ArrayList<Produto>` contendo ao menos um `ProdutoFisico` e um `ProdutoDigital`. Percorra a lista chamando `mostrar()` em cada elemento. O que você observa na saída? Qual conceito isso demonstra?
- t) **e)** Explique com suas palavras: por que é possível guardar objetos de `ProdutoFisico` e `ProdutoDigital` numa lista do tipo `ArrayList<Produto>`?

 **Conceito-chave:** Polimorfismo permite que objetos de tipos diferentes respondam ao mesmo método (`mostrar()`) de formas distintas. A JVM decide em tempo de execução qual versão chamar — isso é chamado de `dynamic dispatch`.

Questão 7 — Classe Principal App

Integração

Na classe `App`, dentro do método `main`, demonstre o funcionamento completo do sistema:

- u) **a)** Instancie um `Usuario` com dados à sua escolha.

- v) **b)** Instancie ao menos um ProdutoFisico e um ProdutoDigital, definindo todos os seus atributos.
- w) **c)** Crie um Pedido, adicione os produtos e chame registrar().
- x) **d)** Exiba no console o resultado de gerarResumo().
- y) **e)** Chame listarProdutos() e observe a saída polimórfica no console.



Questão 8 — Reflexão Teórica

Conceitos e Justificativas

Responda com base no código implementado:

- z) **a)** Qual pilar da POO é representado pelo uso de protected nos atributos de Pessoa? Justifique.
- aa) **b)** Qual é a diferença entre herança e composição? Identifique um exemplo de cada um no sistema.
- bb) **c)** O método removerProduto aparece duas vezes com assinaturas diferentes. Como se chama esse recurso e qual é sua utilidade?
- cc) **d)** Qual a diferença entre sobrecarga (overloading) e sobrescrita (overriding)? Identifique um exemplo de cada no sistema implementado.
- dd) **e)** A classe Pedido implementa Registravel. O que aconteceria se você declarasse uma variável do tipo Registravel e atribuísse um objeto Pedido a ela? Que métodos estariam acessíveis?



Dica Final:

A Questão 6 exige ajustes na classe Produto — ao torná-la abstrata, revise o impacto nas classes anteriores. Implemente e compile de forma incremental para identificar erros mais facilmente.